

Conjunto de Mandelbrot com Paralelismo

Disciplina: Infraestrutura de Software · Implementação 2

Aluno: Mateus Reinaux Batista Meira · **Login:** mrbm

Repositório: <https://github.com/eFlow-hub/mandelbrot>

Data: 31 de agosto de 2026

1. Objetivo

Gerar a imagem do conjunto de Mandelbrot em C por quatro caminhos diferentes — uma implementação serial, uma com OpenMP e duas com Pthreads usando estratégias distintas de divisão do trabalho — e comparar o desempenho entre elas. As quatro produzem a mesma imagem; o que muda é como o trabalho é repartido entre as threads.

O programa recebe `largura`, `altura`, `max_iteracoes` e `num_threads`, grava quatro arquivos de imagem e um `times.txt` com o tempo de cada implementação, numa única execução.

2. Ambiente

Item	Valor
Sistema	WSL2 Ubuntu, kernel 6.6.87.2-microsoft-standard-WSL2
Compilador	gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Flags	<code>-Wall -Wextra -O2 -fopenmp -pthread</code>
Processador	Intel Core i7-13620H
Núcleos	8 físicos, 16 threads lógicas (2 por núcleo)

3. A especificação decodificada a partir dos gabaritos

O enunciado descreve o cálculo em prosa, mas não fixa duas decisões que mudam o resultado byte a byte: como converter o índice do pixel em coordenada do plano, e se a normalização arredonda ou trunca. Como a correção compara os arquivos exatamente, escolher diferente reprova o teste mesmo com a matemática correta.

Em vez de tentar e errar, as fórmulas foram deduzidas dos três arquivos de teste fornecidos, antes de escrever o código.

3.1 Mapeamento do pixel para o plano complexo

```
c_re = -2.0 + x * (3.0 / largura);
c_im = -1.5 + y * (3.0 / altura);
```

O divisor é `largura`, não `largura-1`. A verificação usa duas propriedades independentes do conjunto. Primeira: no teste 1 (4 4 50 1) a terceira linha do gabarito é 255 255 255 255. Com esse mapeamento, a linha `y=2` cai em `c_im = 0`, isto é, sobre o eixo real, e os quatro pontos têm parte real `-2,0`, `-1,25`, `-0,5` e `0,25` — todo o intervalo real `[-2; 0,25]` pertence ao conjunto, então os quatro saturam em 255. Segunda: no teste 2 (6 6 30 3) as linhas 1 e 5 são idênticas, assim como as linhas 2 e 4. É a simetria conjugada do conjunto em torno do eixo real, que só cai nos índices certos com esse passo.

3.2 Laço de escape

```
int iter = 0;
double zr = 0.0, zi = 0.0;

while (zr*zr + zi*zi <= 4.0 && iter < max_iter) {
    double t = zr*zr - zi*zi + c_re;
    zi = 2.0*zr*zi + c_im;
    zr = t;
    iter++;
}
```

O critério de divergência é $|z| > 2$, aplicado aqui como $|z|^2 > 4$ para evitar a raiz quadrada — este é o trecho mais executado do programa inteiro, e eliminar o `sqrt()` é a otimização mais barata disponível. O teste é avaliado antes de cada iteração; mudá-lo para `< 4.0` ou movê-lo para o fim do laço desloca toda a contagem em uma unidade.

A variável temporária `t` não é estilo: o novo `zi` depende do `zr` antigo. Atribuir `zr` primeiro corrompe o cálculo silenciosamente.

Verificação pontual: para `c = 0,5` a sequência é $0,5 \rightarrow 0,75 \rightarrow 1,0625 \rightarrow 1,6289 \rightarrow 3,153$, escapando em `iter = 5`. O gabarito do teste 2 traz 42 nessa posição, e $(5 \times 255) / 30 = 42$.

3.3 Normalização

```
pixel = (iter * 255) / max_iter; /* divisão inteira: trunca */
```

O caso decisivo está no teste 2: com `max_iter = 30` e `iter = 1`, o valor exato é 8,5. Truncar dá 8, arredondar daria 9, e o gabarito traz 8. Pontos do conjunto atingem `iter == max_iter` e resultam em 255 sem caso especial.

3.4 Formato de saída

Apesar da extensão `.pgm`, o arquivo não tem cabeçalho de formato: apenas os valores de intensidade, separados por um espaço, uma linha de texto por linha da imagem, sem espaço à direita. Confirmado por inspeção hexadecimal do gabarito.

4. Arquitetura

A peça central é a função que calcula uma linha inteira:

```
static void calcula_linha(unsigned char *linha, int y, const Config *cfg);
```

A linha é a unidade de trabalho que as quatro implementações repartem. Como o valor de um pixel não depende de nenhum outro, e cada thread escreve num conjunto disjunto de posições do buffer, **nenhuma sincronização é necessária para escrever a imagem**. Uma condição de corrida exige acesso concorrente ao mesmo endereço com pelo menos uma escrita, e isso nunca acontece aqui.

O problema é, portanto, *embaraçosamente paralelo*: a dificuldade não está em coordenar as threads, e sim em *dividir* o trabalho — que é exatamente o que as duas estratégias de Pthreads exploram.

A imagem é um bloco linear contíguo de `largura × altura` bytes, com a linha `y` começando em `img + y * largura`. A alternativa, um vetor de ponteiros para linhas, exigiria `altura + 1` alocações em vez de uma e espalharia as linhas pelo heap, prejudicando a localidade de cache — inaceitável num programa cujo propósito é ser rápido. Cada pixel é `unsigned char`, que é exatamente a faixa `[0, 255]` do valor normalizado.

As quatro implementações são registradas numa tabela percorrida pela `main`, que mede, calcula, grava o arquivo e acumula o tempo. Acrescentar uma implementação é acrescentar uma linha.

4.1 Serial

Percorre as linhas em ordem. Vale 5% da nota, mas seu papel real é ser o oráculo de correção: os outros três são validados por comparação com ele.

4.2 Pthreads 1 — divisão estática por faixas contíguas

Cada thread recebe um intervalo `[y_ini, y_fim)` antes de começar e nunca mais consulta estado compartilhado. Zero sincronização no laço, e as linhas de uma mesma thread ficam vizinhas na memória.

O resto da divisão inteira é distribuído entre as primeiras `altura % num_threads` threads. Sem isso, `altura = 6` com 4 threads daria $6/4 = 1$ e quatro faixas de uma linha cobririam apenas 4 das 6 linhas — que é precisamente o caso do teste 3 oficial (10 6 40 4).

4.3 Pthreads 2 — fila dinâmica de linhas

Um único contador compartilhado entrega a próxima linha livre a quem terminar. Quem pega uma linha barata volta logo; quem pega uma cara processa menos linhas no total. A carga se equilibra sozinha, sem ninguém precisar conhecer o custo de cada linha de antemão.

O incremento fica sob mutex porque `proxima++` compila para ler, somar e escrever. A região crítica contém *apenas* o incremento: o cálculo fica fora dela, senão as threads se serializariam e a versão paralela pagaria o custo do lock sem ganho nenhum.

4.4 OpenMP

```
#pragma omp parallel for schedule(dynamic) num_threads(cfg->num_threads)
for (y = 0; y < cfg->altura; y++)
    calcula_linha(img + (size_t)y * cfg->largura, y, cfg);
```

A escolha de `schedule(dynamic)` segue a mesma lógica da estratégia 2. Com o `schedule(static)` padrão, o OpenMP repartiria em blocos iguais e sofreria o mesmo desbalanceamento da estratégia 1 — o que a seção 7 mostra medido.

Nenhuma cláusula `private` é necessária: `zr`, `zi` e `iter` são locais de `intensidade()`, chamada de dentro da região paralela, e variáveis locais de função vivem na pilha da thread que a executa.

4.5 Tratamento de erros

Todos os parâmetros são lidos com `strtol`, verificando onde a leitura parou (`endptr`) e se houve estouro de faixa (`errno`), e exigindo inteiro estritamente positivo. O produto `largura × altura` é conferido contra `SIZE_MAX` antes do `malloc`. Falhas de alocação, de criação de arquivo e de `pthread_create` produzem mensagem em `stderr` e encerram com status 1. No caminho de erro de criação de threads, as já criadas são aguardadas antes de liberar as estruturas que elas ainda usam.

5. Testes

O alvo `make check` executa os três casos oficiais e compara *todos* os arquivos `.pgm` gerados contra o mesmo gabarito. Isso cobre as duas exigências do enunciado com uma verificação só: cada implementação bate com o esperado, e as quatro são idênticas entre si. Verifica ainda que nada é impresso em `stdout` e que o `times.txt` é gerado.

```
==== ./mandelbrot 10 6 40 4 ====
OK: mandelbrot_mrbm_openmp.pgm
OK: mandelbrot_mrbm_pthreads1.pgm
OK: mandelbrot_mrbm_pthreads2.pgm
OK: mandelbrot_mrbm_serial.pgm
==== stdout mudo ====
OK: stdout vazio

TODOS OS TESTES PASSARAM
```

Além dos gabaritos, foram verificados 13 casos de erro (argumentos faltando, sobrando, não numéricos, zero, negativos, acima de `INT_MAX`, string vazia, `"3x"`), todos rejeitados com mensagem coerente e status 1. Em determinismo: imagem 1×1; 11 threads para 5 linhas (mais threads que linhas); e 200×200 com 16 threads repetido 10 vezes — em todos os casos os quatro arquivos saíram idênticos.

6. Experimentos: os erros e o que cada um ensinou

Sete erros foram reproduzidos deliberadamente em programas isolados, no diretório `experimentos/`. Cada um roda nos modos `bug` e `correto`, para o contraste ficar no mesmo binário. As saídas completas estão em `evidencias.log`.

6.1 Mapeamento e normalização

As duas alternativas plausíveis descartadas na seção 3 falham de maneiras opostas. Dividir por `largura-1` erra três das quatro linhas do teste 1: a linha de 255 migra de `y=2` para `y=0` e a simetria desaparece. Arredondar em vez de truncar erra dois pixels, cada um por uma unidade — a imagem fica visualmente idêntica ao gabarito.

O contraste é a lição: o primeiro erro qualquer inspeção visual pegaria; o segundo só a comparação byte a byte pega. Se a validação fosse "abrir a imagem e ver se parece o Mandelbrot", o segundo chegaria intacto à entrega — e custaria a nota das quatro implementações de uma vez, já que todas compartilham a mesma função de intensidade.

6.2 O argumento passado ao `pthread_create`

Passar `&i` em vez de `&ids[i]` faz todas as threads apontarem para a mesma variável, que o laço continua incrementando. Em cinco execuções, cinco resultados diferentes:

```
resumo: 4 indice(s) repetido(s), 4 ausente(s)
resumo: 2 indice(s) repetido(s), 2 ausente(s)
resumo: 2 indice(s) repetido(s), 2 ausente(s)
resumo: 2 indice(s) repetido(s), 3 ausente(s)
resumo: 2 indice(s) repetido(s), 3 ausente(s)
```

Cada índice ausente é uma faixa de linhas que ninguém calcula.

6.3 O resto da divisão inteira

Nas dimensões do teste 3 oficial, sem tratar o resto:

```
modo: bug | base=1 resto=2 | linhas cobertas: 4 de 6

@@ diff contra o gabarito @@
-6 19 19 31 38 255 255 255 44 19
-6 12 19 19 19 25 44 25 19 12
+0 0 0 0 0 0 0 0 0 0
+0 0 0 0 0 0 0 0 0 0
```

As duas últimas linhas saem com o conteúdo inicial do buffer. Um caso de teste cujas dimensões não dividem exatamente pelo número de threads vale mais que dez com dimensões redondas.

6.4 O contador da fila sem mutex

Com 20 000 linhas e 8 threads, o ThreadSanitizer acusa a corrida sobre a variável `proxima`, e a contagem mostra o efeito:

```
modo: bug (sem mutex)
linhas nunca processadas : 0
linhas processadas 2+ vez: 3067
total de processamentos : 24379 (esperado 20000)

modo: correto (com mutex)
linhas processadas 2+ vez: 0
total de processamentos : 20000 (esperado 20000)
```

Achado que contraria a previsão. Esperavam-se linhas puladas e duplicadas; a medição mostra zero puladas e milhares duplicadas. O motivo é que um incremento não atômico só pode *perder* atualizações, nunca ganhar: quando duas threads leem o mesmo valor `v`, ambas escrevem `v+1`, o contador avança 1 em vez de 2, a linha `v` sai duas vezes e o fim da fila demora mais a chegar — então nada fica sem processar.

A consequência prática é contraintuitiva: no Mandelbrot real a imagem sairia **correta**, porque recalculer uma linha grava exatamente os mesmos bytes. O prejuízo é trabalho desperdiçado — 22% a mais nesta medição —, não imagem errada. A corrida continua sendo comportamento indefinido, e uma formulação ligeiramente diferente (testar `proxima < altura` antes de incrementar) produziria linhas puladas de fato.

6.5 OpenMP com variáveis compartilhadas

Declarando `zr`, `zi` e `iter` fora da região paralela, cinco execuções produzem cinco imagens diferentes. Não trava e não acusa erro: apenas devolve um resultado diferente a cada vez.

Segundo achado inesperado. Compilado com `-O2`, o bug *não se manifesta*: as cinco execuções dão o mesmo resultado, e igual ao da versão correta. O compilador mantém as variáveis compartilhadas em registradores dentro do corpo do laço, e o entrelaçamento deixa de ser observável. A corrida continua no código-fonte; só o nível de otimização esconde o sintoma. É a demonstração de que "testei e funcionou" não é prova de ausência de corrida — a evidência precisou ser colhida com `-O0`.

6.6 atoi contra strtol

entrada abc	atoi=0	strtol=0	REJEITADO
entrada 3x	atoi=3	strtol=3	REJEITADO
entrada 999999999	atoi=1410065407	strtol=999999999	REJEITADO

O caso "999999999" é o mais grave: `atoi` devolve 1 410 065 407, um número perfeitamente plausível, sem nenhum sinal de que a conversão estourou. No programa real, `num_threads = 0` produziria imagem vazia e `max_iteracoes = 0` produziria divisão por zero.

6.7 clock() contra clock_gettime()

1 thread(s):	clock()= 2.1897s	clock_gettime()= 2.1902s	razao= 1.00x
2 thread(s):	clock()= 2.1615s	clock_gettime()= 1.0817s	razao= 2.00x
4 thread(s):	clock()= 2.1372s	clock_gettime()= 0.5371s	razao= 3.98x
8 thread(s):	clock()= 2.2245s	clock_gettime()= 0.2784s	razao= 7.99x
16 thread(s):	clock()= 2.6048s	clock_gettime()= 0.1639s	razao=15.89x

`clock()` permanece praticamente constante em 2,2 s — é o total de CPU consumida, que não muda quando o trabalho é dividido — enquanto o tempo real cai de 2,19 s para 0,16 s. A razão entre os dois acompanha o número de threads quase exatamente. Com uma thread os dois coincidem, e é por isso que o erro passa despercebido enquanto só existe a implementação serial: **é um bug que só nasce quando o contexto muda.**

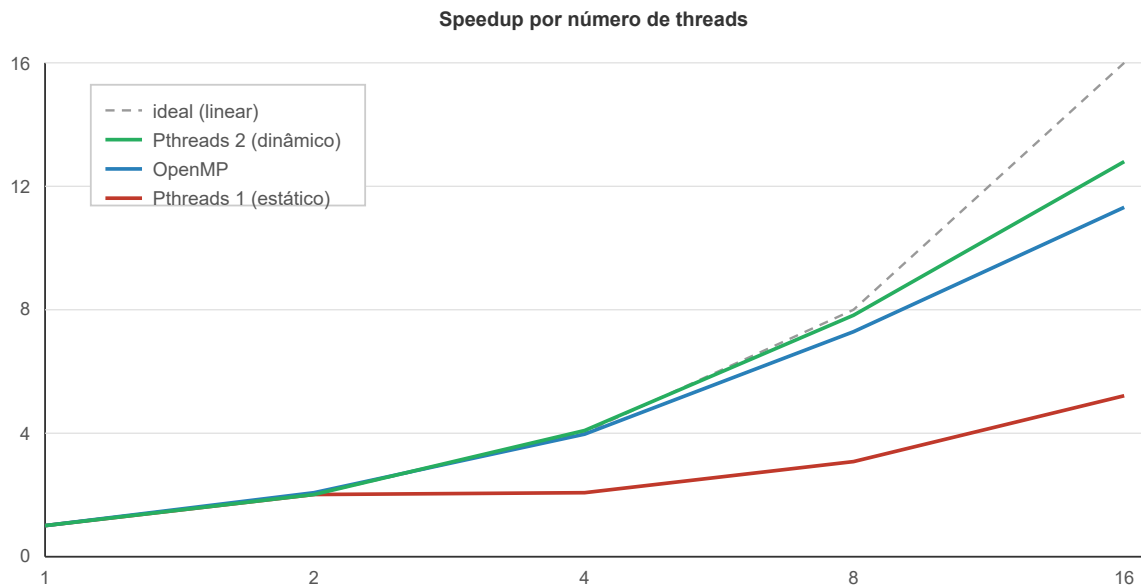
Erro de método corrigido durante o próprio experimento. A primeira versão media o tempo de não fazer nada. O buffer era `static` e nunca lido, então o `gcc -O2` concluiu que as escritas eram mortas e removeu o laço de cálculo inteiro — o assembly gerado tinha 2 instruções de ponto flutuante no programa todo, e 1200×1200 pixels "levavam" 48 microssegundos. Foi corrigido tirando o `static` e somando os pixels ao final.

7. Desempenho

Imagem 1200×1200, `max_iter = 1000`, duas execuções por configuração. Os tempos abaixo são a média. O benchmark foi executado no sistema de arquivos nativo do Linux: rodando em `/mnt/c`, o custo de gravar quatro arquivos através do sistema de arquivos do Windows domina e inviabiliza a comparação.

Tempo serial de referência: **1,093 s**.

Threads	OpenMP		Pthreads 1 (estático)		Pthreads 2 (dinâmico)	
	tempo	speedup	tempo	speedup	tempo	speedup
1	1,078 s	1,01×	1,092 s	1,00×	1,091 s	1,00×
2	0,533 s	2,05×	0,536 s	2,04×	0,539 s	2,03×
4	0,276 s	3,96×	0,523 s	2,09×	0,267 s	4,09×
8	0,149 s	7,32×	0,358 s	3,05×	0,139 s	7,84×
16	0,097 s	11,30×	0,209 s	5,22×	0,085 s	12,80×



Eixo horizontal: número de threads. Eixo vertical: speedup sobre a versão serial.

7.1 Estático contra dinâmico

Até 2 threads as três curvas coincidem. A partir de 4 elas se separam, e a diferença cresce: com 16 threads a estratégia dinâmica atinge 12,8× enquanto a estática empaca em 5,2× — menos da metade.

A causa é a geometria do conjunto. A região `[-1,5; 1,5]` no eixo imaginário coloca o corpo do conjunto no meio da imagem, e as linhas que o cruzam custam `max_iter` iterações por pixel, enquanto as das bordas escapam em uma ou duas. Com faixas contíguas, a thread que recebe a faixa central faz várias

vezes mais trabalho que as demais, e o tempo total é o da mais lenta — as outras terminam e ficam ociosas.

O caso de 4 threads é o mais ilustrativo: 0,523 s é praticamente o mesmo tempo de 2 threads (0,536 s). Dobrar o número de threads não trouxe ganho nenhum, porque a faixa mais cara continuou custando o mesmo.

A fila dinâmica não precisa saber nada disso: quem termina cedo simplesmente pega mais trabalho. O custo do mutex é irrelevante porque ele é adquirido uma vez por linha, enquanto o cálculo de uma linha custa milhares de iterações.

7.2 Threads lógicas contra núcleos físicos

Previsão registrada antes da medição, e parcialmente refutada. A expectativa anotada no início era que o speedup crescesse até 8 threads (os núcleos físicos) e depois achatasse, já que o *hyperthreading* ajuda sobretudo quando há esperas por memória e o Mandelbrot é aritmética pura.

A medição desmente: de 8 para 16 threads o Pthreads 2 vai de 7,84× para 12,80×, um ganho adicional de 63%. A explicação é que o laço de escape tem uma cadeia de dependências serial — cada iteração precisa do resultado da anterior — e a latência das operações de ponto flutuante deixa bolhas no pipeline que o segundo contexto do núcleo consegue preencher. O ganho é real, embora abaixo do dobro que 16 threads sugeririam.

7.3 Quando o paralelismo custa mais do que rende

Na imagem 6×6 do teste 2, os tempos se invertem:

```
Serial: 0.000002s  OpenMP: 0.000248s  Pthreads1: 0.000449s  Pthreads2: 0.000357s
```

As versões paralelas são cem vezes mais lentas. Criar e sincronizar threads custa dezenas de microssegundos, e o cálculo de 36 pixels custa 2 microssegundos. É o mesmo fenômeno visível no `times.txt` de referência da disciplina, onde o OpenMP aparece muito mais lento que o serial. Paralelizar só compensa quando o trabalho por thread supera o custo de criá-la.

8. Dificuldades

1. **Fixar as fórmulas exatas.** Foi a parte que mais exigiu análise. A decisão de deduzi-las dos gabaritos antes de programar, em vez de tentar e corrigir, evitou um ciclo longo de tentativa e erro sobre três decisões independentes.
2. **Reproduzir corridas de dados de forma confiável.** Duas resistiram: a do OpenMP só aparece com -00, e a do contador produziu um sintoma diferente do previsto. Ambas exigiram investigar em vez de aceitar o primeiro resultado.
3. **Medir o que se pretende medir.** Dois erros de método apareceram antes de qualquer número confiável: `clock()` medindo CPU somada, e o compilador removendo o laço inteiro de um benchmark cujo resultado não era lido.

4. **I/O do sistema de arquivos montado.** A primeira tentativa de benchmark foi abortada após dez minutos sem terminar, porque gravar quatro imagens de 16 MB por execução através do `/mnt/c` domina completamente o tempo.

9. Limitações conhecidas

- Os parâmetros são inteiros positivos até `INT_MAX`; a região do plano é fixa, sem opção de recorte ou zoom.
- A imagem inteira é mantida em memória, o que limita as dimensões à RAM disponível. Um esquema por blocos com escrita incremental removeria o limite, ao custo de complicar a divisão do trabalho.
- A simetria conjugada do conjunto não é explorada. Calcular metade das linhas e espelhar reduziria o trabalho quase pela metade, mas mascararia a comparação entre estratégias de divisão, que é o objetivo do exercício.
- O tamanho do lote da fila dinâmica é de uma linha. Para imagens muito estreitas, um lote maior amortizaria melhor o custo do mutex.

10. Conclusão

As quatro implementações produzem arquivos idênticos e passam nos três casos oficiais. A comparação entre elas confirma que, num problema embaraçosamente paralelo mas com carga desbalanceada, **a estratégia de divisão importa mais que o mecanismo de paralelização**: OpenMP e Pthreads dinâmico, que usam abordagens equivalentes de distribuição, entregam desempenho equivalente (11,3× e 12,8×), enquanto o Pthreads estático, com o mesmo mecanismo do dinâmico, fica em 5,2×.

Dos sete erros instrumentados, os dois mais instrutivos foram justamente os que contrariaram a expectativa: a corrida do contador que degrada desempenho sem corromper a imagem, e a corrida do OpenMP que a otimização do compilador torna invisível. Ambos reforçam a mesma lição — a ausência de sintoma não é prova de correção em código concorrente.

11. Repositório e reprodução

<https://github.com/eFlow-hub/mandelbrot>

```
make                # compila
make check          # roda os tres gabaritos oficiais
make clean          # remove binarios e saidas
./mandelbrot 1200 1200 1000 8
```

O histórico segue commits atômicos alinhados às seções deste relatório: um por bloco de implementação e um por experimento, cada um com a justificativa na mensagem. O arquivo `evidencias.log` traz as saídas brutas de todos os testes e experimentos, com os comandos que as produziram.